

LangChain Agent vs. Raw Python Agent

Week 5 – Day 2

Overview

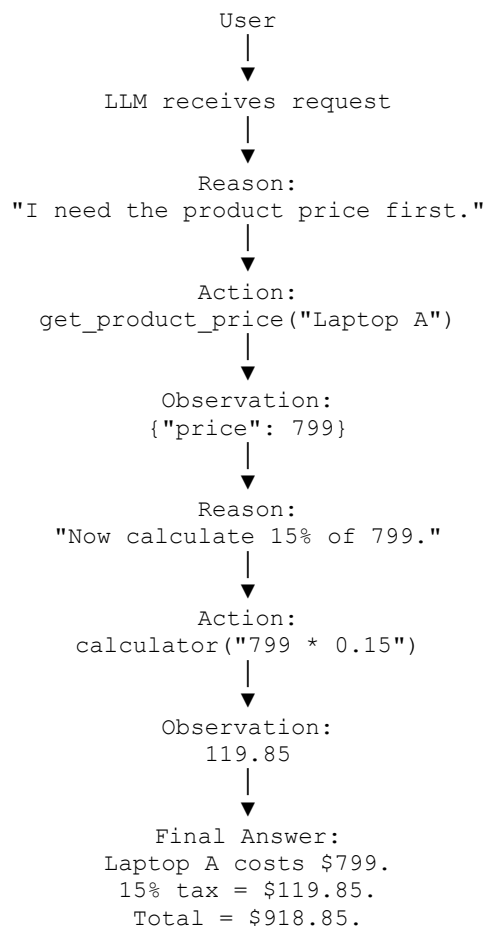
The objective of this task was to rebuild the raw Python agent from Day 1 using LangChain while preserving the same core ReAct workflow (Reason → Act → Observe). Instead of manually implementing the reasoning loop, tool schemas, memory, and message handling, LangChain provides reusable abstractions that significantly reduce boilerplate code while keeping the overall agent behavior the same.

Comparison

Raw Python Agent	LangChain Agent
Manual API calls using OpenAI client	ChatOpenAI wrapper
Hand-written JSON schemas for tools	@tool decorator generates schemas automatically
Custom reasoning loop using while	AgentExecutor manages the loop
Manual conversation history	RunnableWithMessageHistory
Manual parsing of model output	Built-in structured output using Pydantic

Annotated Reasoning Trace

User: "What is the price of Laptop A and what would a 15% tax add?"



Unlike the raw Python implementation, LangChain hides the internal reasoning step. With `verbose=True`, only the tool invocations and observations are displayed, while the reasoning itself remains internal to the model.

Memory

Conversation memory was implemented using `RunnableWithMessageHistory`, allowing the agent to remember previous interactions.

Example conversation:

1. What is the price of Laptop A?
2. Now compare it to Laptop B.
3. Which one should I recommend to a budget-conscious client?

Because the earlier conversation was stored in memory, the third response was generated using previous context instead of asking for the product names again.

Structured Output and Error Handling

Structured output was implemented using a Pydantic model, ensuring the final recommendation always contained validated fields such as the recommended product, price, reason, and whether it was budget-friendly.

For error handling, a `get_stock` tool was created that raises a `ToolException` when an unknown product is requested. Since the installed LangChain version (0.3.30) does not support `handle_tool_error` in the `@tool` decorator, graceful recovery was implemented by catching the exception around the agent invocation with a `try/except` block. This prevented the program from terminating unexpectedly and allowed a clear error message to be displayed.

Reflection

Compared with the raw Python implementation, LangChain greatly simplified agent development by providing built-in abstractions for tool registration, conversation memory, structured outputs, and agent execution. This removed a significant amount of boilerplate code and allowed the focus to remain on agent behavior rather than infrastructure.

At the same time, some implementation details became less visible. The reasoning loop is managed internally by `AgentExecutor`, making debugging more difficult than in the manually implemented agent. Additionally, framework-specific conventions, such as raising `ToolException` for tool failures, must still be understood to achieve reliable behavior.